



Predicting IT Support SLA Breaches

Using Logistic Regression and XGBoost on Support Ticket Data

Ashley Barone | Western Governors University



Introduction

Ashley Barone

MSDA Capstone Candidate

- Based in Atlanta, GA
- Background in Internal Auditing, Quality Assurance, and Training & Development
- Bachelor of Science - Computer Science

Why this project?

I am passionate about improving processes, increasing efficiency, and equipping employees with tools that prevent burn out.



The Problem

Service Level Agreements (SLAs) set the timeframe for resolving support tickets.

When SLAs are missed:



Performance Suffers

Teams miss their goals and overall performance drops.



Customers Lose Trust

Satisfaction suffers when promises aren't kept.



Teams Burn Out

Constant reactive problem-solving leaves agents overwhelmed and overworked.

Most support teams today address tickets reactively as they come in, relying mostly on incoming priority flags.



The Goal

Can we create a tool to predict which tickets will breach SLA
before they breach?

If yes, support teams can:



Flag at-risk tickets early



Allocate resources proactively



Improve overall SLA compliance



The Data

10,000

Support tickets analyzed

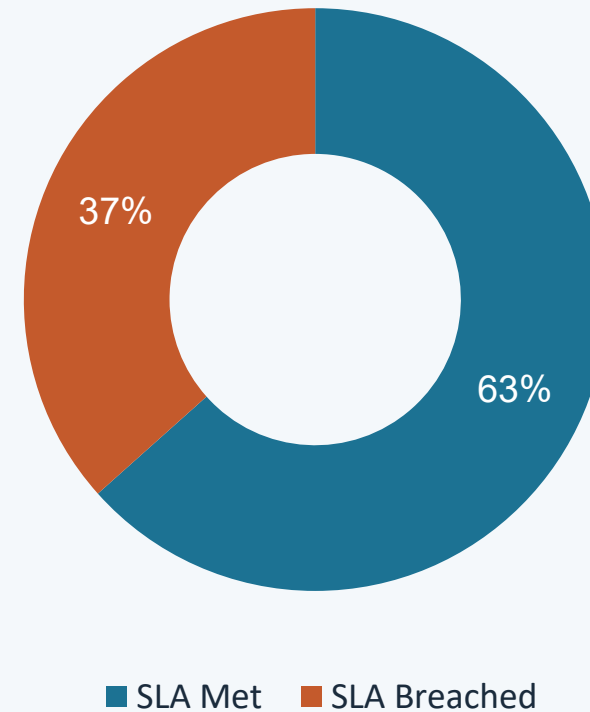
14

Data points per ticket

36.6%

Tickets that breached SLA

Distribution of SLA Outcomes



Source: Kaggle IT Support Tickets dataset (publicly available)



The Data, continued

What can we learn?

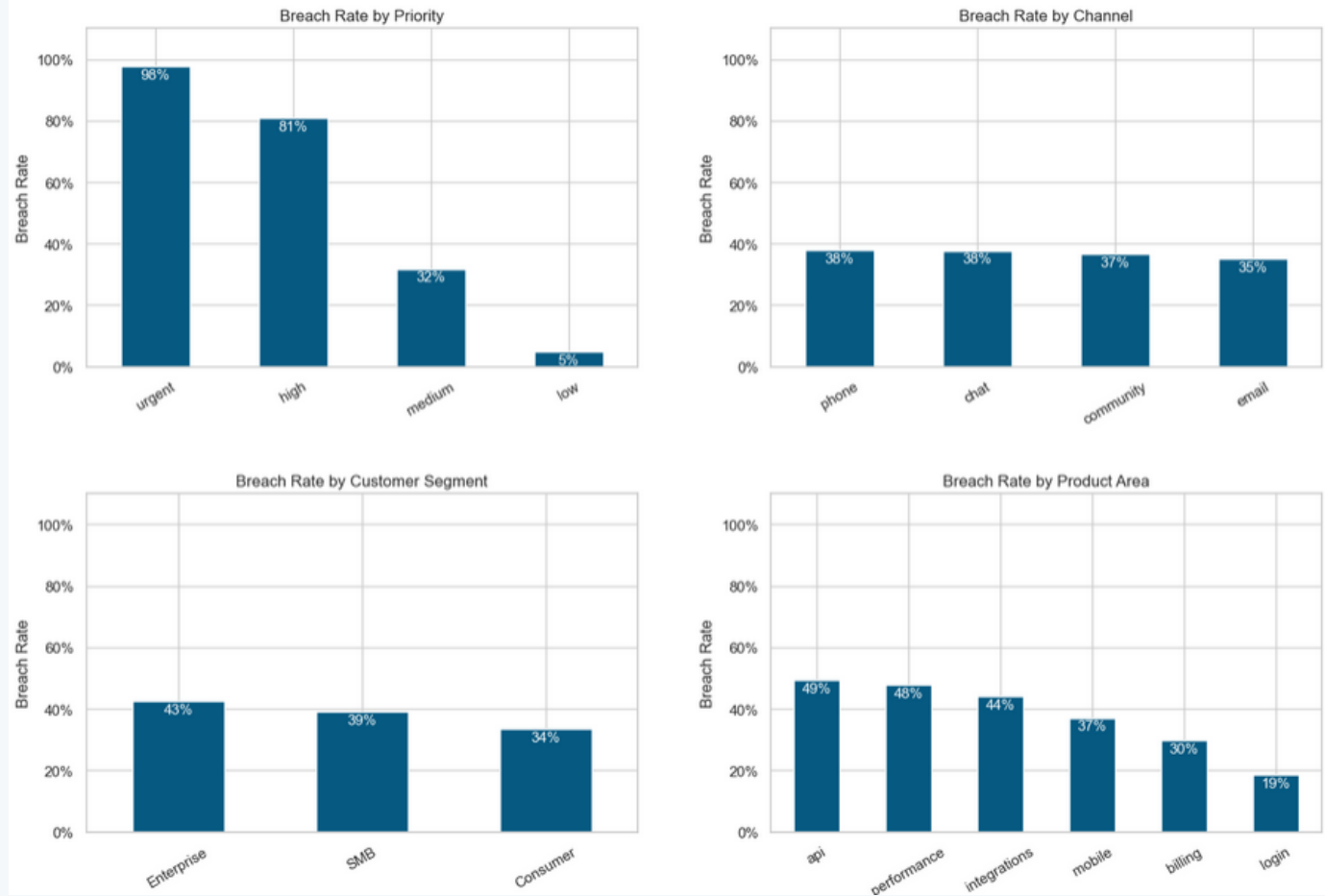
Tickets belong to different categories like priority (low or high), channel (phone or chat), customer segment (enterprise or consumer), and product area (mobile or billing). Exploring how those categories affect SLA outcomes can help inform teams what to look out for and how to allocate resources across categories.

*Exploring the data
before modeling can
help us figure out what
to pay attention to later.*



Source: Kaggle IT Support Tickets dataset (publicly available)

SLA Breach Rate by Categorical Predictors





What's in a Support Ticket?

Each ticket included details across three main categories:



About the Customer

- Customer segment (Consumer, SMB, Enterprise)



About the Ticket

- Product area (login, billing, mobile, etc.)
- Priority level (low, medium, high, urgent)
- Channel (email, phone, chat, community)
- Ticket sentiment
- Tenure of the agent who was assigned the ticket



About the Resolution

- Time to first response
- Time to resolution
- Whether SLA was met
- Customer satisfaction afterward
- If a ticket was escalated
- How many times a ticket was reopened
- Whether a bug was confirmed

Not all of the information in a support ticket will be useful for predicting at-risk tickets. Let's see why.



What I Looked At

Six factors available at ticket creation:

1

Customer Segment

Consumer, SMB, or Enterprise

2

Product Area

Login, billing, mobile, etc.

3

Priority Level

Low, medium, high, urgent

4

Channel

Email, phone, chat, community

5

Agent Experience

Tenure of assigned agent

6

Ticket Sentiment

Tone of the ticket message

Other ticket details (resolution time, customer satisfaction score, escalation status) were left out. Details not known at ticket creation won't help with predicting early breach risks.



A Smart Choice: Avoiding Data Leakage

Some ticket details would give the model an unfair head start.

Used in the model

Known when a ticket comes in

- Customer segment
- Product area
- Priority level
- Channel
- Agent experience
- Ticket sentiment

Excluded from the model

Only known after resolution

- Time to first response
- Time to resolution
- Customer satisfaction score
- Whether ticket was escalated
- Whether ticket was reopened
- Whether a bug was confirmed

Using post-resolution details would make the model look more accurate than it is. We don't want the model to "cheat".



How I Analyzed It

Two prediction models built and compared:

Logistic Regression

Easy, interpretable

A straightforward approach that's good at explaining why it makes the predictions it does. Shows the direction and size of each factor's effect on the target: SLA outcomes.

XGBoost

Complex, validating

A more complex approach. Used to double-check the logistic regression results from a different angle, measuring each factor's contributing importance to the target: SLA outcomes.

When two different methods agree, the results are stronger!



Preparing the Data

Before training, the data needed some cleanup:

1

Convert categories to yes/no values

Models need numbers to make calculations and predictions. Categories like "Email" or "Enterprise" had to be converted 1s and 0s that the model could understand. This is called One-Hot Encoding.

Ex: Priority is converted into four yes/no columns (low, medium, high, and urgent) where a value of 1 in the column means, "yes, this ticket is that priority".

2

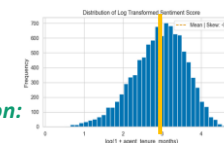
Smooth out unbalanced data

There were more agents with shorter tenure leading to data that is lopsided or skewed. This data needed to be transformed to appear more balanced. The process used is called log-transformation.

Before transformation:



After transformation:



3

Check for overlap between factors

Before building the models, check to make sure no two factors were measuring the same thing. The Variance Inflation Factor gives each variables a score. If that score is less than 10, we can keep it.

Feature	VIF
agent_tenure_months_log	8.5885
priority_low	2.5963
priority_medium	2.4097
product_area_login	2.2991
channel_email	2.1220
product_area_billing	2.1124
product_area_performance	2.0830
product_area_mobile	1.9834
product_area_integrations	1.9773
customer_segment_SMB	1.5215
channel_phone	1.5021
channel_community	1.2980
customer_segment_Enterprise	1.2630
priority_urgent	1.2541
sentiment_score	1.2364

4

Split into training and testing

80% of the data was used to teach the model what to look for when flagging high-risk tickets. The other 20% of the data was used test the model using tickets it had never seen before. Using tickets the model has never seen before ensures that the model is not "cheating" by just remembering the answers.



Did It Work?

Goal: 70% accuracy minimum

Logistic Regression

87.3%

accuracy on the test set

XGBoost

86.7%

accuracy on the test set

Both models far exceeded the goal and agreed!



What Predicts a Breach?

The Logistic Regression model tells us how much and in what way ticket characteristics affect SLA outcomes:

↑ Increases breach risk

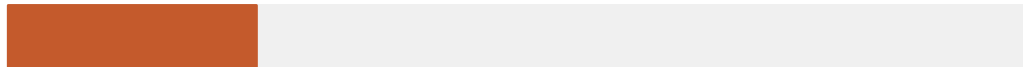
Urgent priority tickets

~11x odds of breach



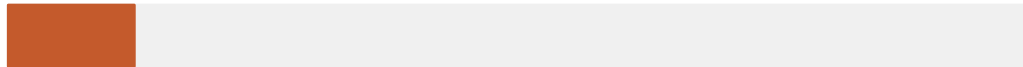
Enterprise customers

~1.8x odds of breach



SMB customers

~1.4x odds of breach



↓ Decreases breach risk

Low priority tickets

~99% lower odds of breach



Positive ticket sentiment

~98% lower odds of breach



Medium priority tickets

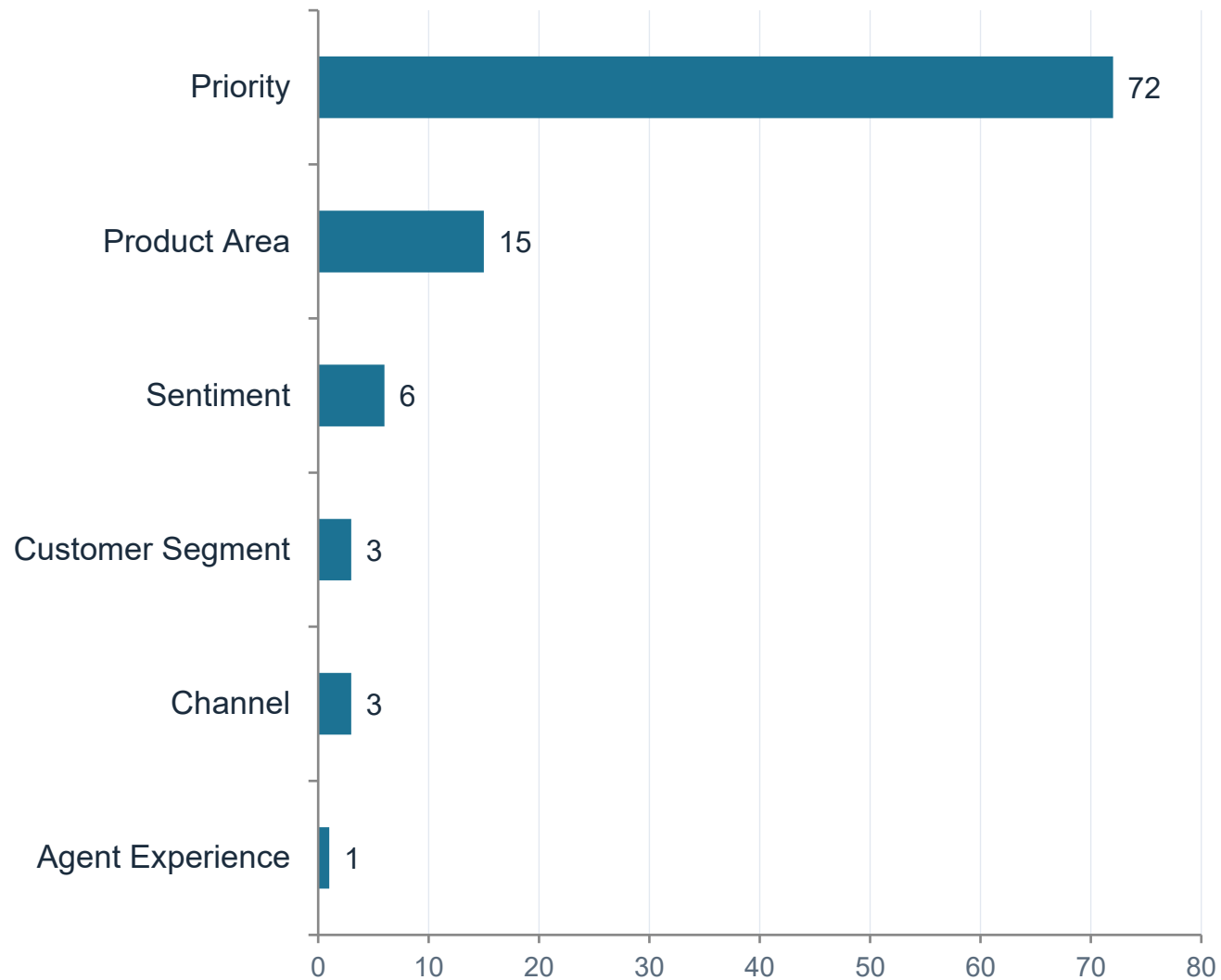
~93% lower odds of breach



Priority drives risk in both directions. Accurate and consistent priority flagging is important.



What Predicts a Breach?



Priority dominates

The strongest predictor driving the majority of risk

Sentiment & product area help

Modest but meaningful contributions to predictions

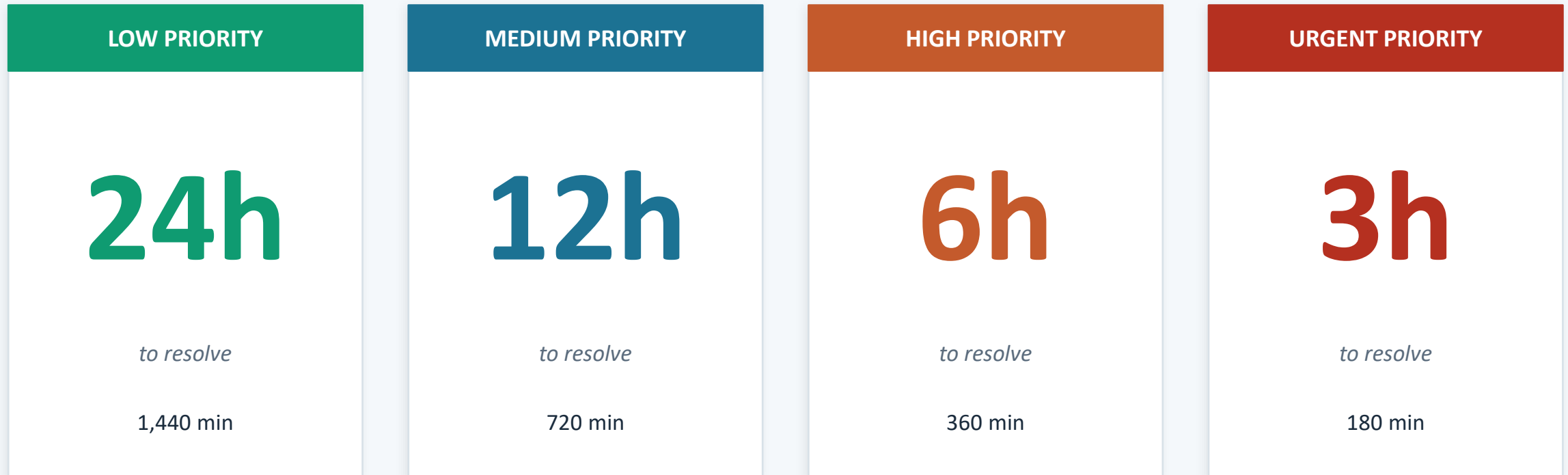
Channel has very little effect

Email vs phone vs chat has almost no effect on SLA outcomes



Why Priority Matters So Much

The dataset revealed probable SLA targets where urgent tickets get a lot less time:



An urgent ticket has 8x less time to resolve than a low-priority one.



What This Means for Support Teams

Use the model to flag at-risk tickets the moment they come in:

~85%

of flagged tickets result in breach

High precision means fewer false alarms

~80%

of all breaches are caught early

Opportunities for proactive action

With this tool, support teams can focus their attention on the tickets that really need it instead of trying to give everything an equal effort.



Limitations & What's Next

Current Limitations

- **Limited customer context**
No age-group, region, or account age data
- **Single-source dataset**
Results may not transfer cleanly to other organizations
- **Logistic Regression is rigid**
Assumes each predictor has a steady, linear affect on the target variable which can miss more complex patterns
- **Use of default model tuning**
Default flagging where values >0.50 result in a flag and values <0.50 do not

Future Directions

- **Add customer demographics**
Industry, region, account age, and more
- **Validate on real organizational data**
Test the model on data with known origins and documentation
- **Test other modeling techniques**
Build and train other types of models to verify the Logistic Regression and XGBoost models' findings
- **Tune flagging sensitivity**
Adjust the model tuning to support team capacity and goals

Limitations help direct meaningful further research.

Thank You!

Ashley Barone

MSDA Capstone | Western Governors University